WHATNEXT VISION MOTORS

Vehicle Management & Test Drive Tracker

Complete Project Documentation

Salesforce Developer Edition | Lightning App, Flows, Apex and Batch Jobs

Part A	Project Documentation	Pages 3 - 10
Part B	Field Creation, Automation, Creation	Pages 11 - 28
Part C	Apex and Trigger Batch Job Coding	Pages 29 - 46

Total pages: 46

Date: 03 Oct 2026

Table of Contents

PART A - PROJECT DOCUMENTATION

1. Introduction, Purpose 3
2. Ideation Phase (Problem Statement, Empathy Map) 4
3. Requirement Analysis (Journey Map, Requirements) 5
3.3 Data Flow and 3.4 Technology Stack 6
4. Project Design (Problem-Solution Fit, Architecture) 7
5. Project Planning and Scheduling (Sprints) 8
6. Project Development (Data and Security Model) 9

PART B - FIELD CREATION, RECORDS AND AUTOMATION

Cover - Field Creation, Automation, Creation 11
1. Overview and 2. Data Model 12
3. How to Create a Custom Field 14
4-7. Vehicle, Dealer, Customer and Lookup Fields 15
8-9. Sample Records Created 19
10. Automation Overview 21
11-12. Flow 1 - Auto Assign Dealer 22
13. Errors and Fixes 24
14-15. Flow 2 - Test Drive Reminder 25
16-17. Testing Checklist and Summary 27

PART C - APEX TRIGGER AND BATCH JOB CODING

Cover - Apex and Trigger Batch Job Coding 29
1-3. Overview, Components, Developer Console 30
4. VehicleOrderTrigger 33
5-7. VehicleOrderTriggerHandler and Stock Logic 34
8. VehicleOrderBatch (Batch Apex) 39
9. VehicleOrderBatchScheduler 42
10-11. Testing, Logs and Debugging 43
12-13. Notes, Improvements and Summary 45

WHATNEXT VISION MOTORS

Vehicle Management & Test Drive Tracker

Browse. Book. Drive. Smarter.

Team Details	
Team ID	: SWTID-2026-4456
Team Leader	: Bhavani S
Team Member 1	: Pooja P
Team Member 2	: Nivetha S
Team Member 3	: Pavithra V

User Story

WhatNext Vision Motors is a Salesforce-based vehicle management solution that lets a dealership network manage its vehicle inventory, dealers, customers, orders, test drives and service requests in one place. The system uses Salesforce custom objects and relationships (Vehicle, Vehicle Dealer, Vehicle Customer, Vehicle Order, Vehicle Test Drive and Vehicle Service Request), a dedicated Lightning App with custom tabs, Flows and Apex for automation, and reports and dashboards for operational visibility. Role-based security keeps customer and dealer data protected.

1. Introduction

1.1 Project Overview

WhatNext Vision Motors is a Salesforce CRM solution built in a Developer Edition org to simplify vehicle sales and after-sales management. It supports the complete lifecycle from vehicle listing and test drive booking through order placement, delivery and service requests. Dealers and staff can manage vehicles, customers and orders, while managers monitor performance through reports and dashboards.

1.2 Purpose

- Centralize vehicle, dealer, customer, order, test drive and service information.
- Simplify test drive booking and vehicle order management.
- Provide a single Lightning App (WhatNext Vision Motors) with tabs for every module.
- Automate order status updates, validations and customer notifications using Flows and Apex.
- Track service requests raised against purchased vehicles.
- Provide secure, role-based access to admins, dealers, sales staff and customers.
- Give managers reports and dashboards on sales, test drives and service performance.

2. Ideation Phase

2.1 Problem Statement (Customer Problem Statement Template)

PS	I am	I'm trying to	But	Because	Which makes me feel
PS-1	Customer	book a test drive easily	booking needs phone calls or visits to the showroom	bookings are not centralized	frustrated
PS-2	Customer	place and track a vehicle order	I do not get a simple view of order progress	order information is scattered	confused
PS-3	Dealer	manage vehicle stock and orders	records are kept manually	there is no centralized system	concerned
PS-4	Customer	raise a service request	service history is difficult to find	service data is not linked to the vehicle	uncertain
PS-5	Admin / Manager	monitor sales and service performance	data is spread across sheets	centralized reporting is missing	concerned

2.2 Empathy Map Canvas - Customer

Think & Feel	Hear	See
Wants an easy way to explore vehicles, book a test drive and place an order. Feels confident when the process is transparent.	Reviews and feedback from friends and family. Offers and announcements from dealers.	Vehicle models (Sedan, SUV, EV), dealer details, order status and service updates.
Say & Do	**Pain**	**Gain**
Books test drives, compares models, places orders, raises service requests.	Repeated calls to the showroom, unclear order status, missing service history.	One place for bookings, orders and service; timely updates; better buying experience.

2.3 Brainstorming & Idea Prioritization

Idea considered	Description	Limitation
Manual registers	Maintain vehicle and customer details on paper.	Error-prone and slow.
Spreadsheet tracking	Track vehicles, orders and test drives in spreadsheets.	Limited collaboration and scalability.
Standalone dealer portal	Develop a separate custom application.	High development and maintenance cost.
Salesforce CRM (selected)	Custom objects, Lightning App, Flows, Apex, reports and dashboards.	Scalable, secure and quick to build.

Final selection: Salesforce CRM, chosen for centralized vehicle management, automation through Flows, role-based security, easy navigation through a Lightning App, and reports and dashboards.

3. Requirement Analysis Phase

3.1 Customer Journey Map

Stage	What happens	Customer thought	Emotion
1. Browse Vehicles	Customer explores available vehicles and models.	Lots of options!	Curious
2. Test Drive Booking	Customer books a Vehicle Test Drive with a dealer.	That was easy to book.	Happy
3. Test Drive	Customer drives the selected vehicle.	I like this one.	Excited
4. Order Placement	A Vehicle Order record is created.	My order is placed.	Confident
5. Order Processing	Dealer confirms, processes and updates order status.	I can see progress.	Reassured
6. Delivery	Vehicle is delivered to the customer.	My new vehicle is here!	Delighted
7. Service Request	Customer raises a Vehicle Service Request.	Help is easy to get.	Satisfied
8. Feedback	Customer shares feedback.	They listened.	Valued

3.2 Solution Requirements

Functional Requirements

FR No	Requirement
FR-1	Vehicle record creation and management (model, type, status)
FR-2	Vehicle model classification using picklist (Sedan, SUV, EV, etc.)
FR-3	Vehicle dealer information management
FR-4	Vehicle customer information management
FR-5	Vehicle order creation and status tracking
FR-6	Test drive booking and scheduling
FR-7	Service request creation and tracking
FR-8	Automated notifications on order and service updates
FR-9	Reports and dashboards
FR-10	Role-based access for admin, dealer and customer

Non-Functional Requirements

NFR	Description
Usability	Intuitive Lightning App with a tab for every object
Security	Profiles, permission sets and field-level security
Reliability	Accurate order, test drive and service information
Performance	Timely Flow and Apex execution
Availability	Salesforce cloud-based access
Scalability	Supports growing vehicles, dealers and customers
Data Integrity	Relationship-based data with validation rules
Maintainability	Flows and configurations can be updated easily

3.3 Data Flow

Vehicle management flow: Customer → Test Drive Booking → Vehicle Test Drive record → Vehicle Order → Order status update by Dealer → Notification → Vehicle Service Request → Reports & Dashboards.

Salesforce data layer: Vehicle__c | Vehicle_Dealer__c | Vehicle_Customer__c | Vehicle_Order__c | Vehicle_Test_Drive__c | Vehicle_Service_Request__c

3.4 Technology Stack

Layer	Technology
Platform	Salesforce Developer Edition
UI	Lightning App (WhatNext Vision Motors), custom tabs, Lightning Record Pages
Database	Salesforce custom objects
Business logic	Salesforce Flows, Apex classes and triggers
Automation	Record-triggered flows, screen flows, scheduled flows
Notifications	Email alerts and email templates
Processing	Batch Apex
Security	Profiles, permission sets, role hierarchy, field-level security, sharing rules
Reporting	Reports and dashboards

4. Project Design Phase

4.1 Problem-Solution Fit

Problem	Solution	Benefit
Test drive booking is difficult	Vehicle Test Drive object and tab	Easy booking
Order tracking is unclear	Vehicle Order object with status field	Transparent progress
Manual updates	Salesforce Flows	Automated processes
Scattered information	Vehicle, Dealer, Customer, Order, Test Drive and Service Request objects	Structured data
Hard to monitor operations	Reports and dashboards	Better visibility
Different users need different access	Profiles, roles and permission sets	Secure access

4.2 Proposed Solution

Parameter	Description
Problem	Vehicle sales, test drive and service data is scattered and hard to manage
Solution	Salesforce-based vehicle management system
Innovation	A single Lightning App with linked objects, automated Flows and Apex
Impact	Centralized records, faster order handling, better customer experience
Scalability	Supports more dealers, vehicles and customers; Experience Cloud and AI can be added later

4.3 Solution Architecture

Layer	Components
Users	Customer, Dealer, Sales staff, Admin
Interface	WhatNext Vision Motors Lightning App; tabs: Vehicles, Vehicle Customer, Vehicle Dealers, Vehicle Orders, Vehicle Test Drives, Vehicle Service Requests
Data layer	Vehicle__c, Vehicle_Dealer__c, Vehicle_Customer__c, Vehicle_Order__c, Vehicle_Test_Drive__c, Vehicle_Service_Request__c
Automation layer	Flows, Apex classes and triggers, Batch Apex, email alerts
Security layer	Profiles, roles, permission sets, sharing rules, field-level security
Insights	Reports and dashboards

5. Project Planning & Scheduling

5.1 Project Planning

- Agile methodology
- Epics, stories and story points
- Sprint-based execution
- Velocity used for estimation

Product Backlog, Sprint Schedule and Estimation

Sprint	Epic	USN	User Story / Task	Pts	Priority
Sprint-1	Developer Setup	USN-1	As an admin, I want to create and configure the Salesforce Developer org so that I can build WhatNext Vision Motors.	3	High
Sprint-2	Data Modeling	USN-2	As an admin, I want to create Vehicle, Dealer, Customer, Order, Test Drive and Service Request objects with fields and relationships so that data is stored systematically.	5	High
Sprint-2	Data Modeling	USN-3	As a staff member, I want custom tabs and a Lightning App so that I can navigate all vehicle modules easily.	5	High
Sprint-3	Automation	USN-4	As a system, I want validation rules and mandatory fields so that incomplete records are prevented.	3	High
Sprint-3	Automation	USN-5	As a sales coordinator, I want Flows to update order and test drive status so that manual effort is reduced.	3	High
Sprint-4	Apex	USN-6	As a system, I want Apex triggers and classes to handle critical vehicle and order actions accurately.	5	High
Sprint-4	Apex	USN-7	As a manager, I want Batch Apex jobs to update records periodically so that data stays current.	5	High
Sprint-5	Reports	USN-8	As a manager, I want reports on orders, test drives and service requests so that I can monitor performance.	4	High
Sprint-6	Dashboards	USN-9	As a manager, I want dashboards showing key metrics so that I can monitor operations efficiently.	4	Medium

Project Tracker, Velocity and Burndown

Sprint	Total points	Duration	Start	Planned end
Sprint-1	20	6 days	05 Oct 2026	10 Oct 2026
Sprint-2	20	6 days	12 Oct 2026	17 Oct 2026
Sprint-3	20	6 days	19 Oct 2026	24 Oct 2026
Sprint-4	20	6 days	26 Oct 2026	31 Oct 2026
Sprint-5	20	6 days	02 Nov 2026	07 Nov 2026
Sprint-6	20	6 days	09 Nov 2026	14 Nov 2026

6. Project Development Phase

Phase 1: Requirement Analysis & Planning

Objective: Customers and dealers often struggle with scattered information when booking test drives, placing vehicle orders and raising service requests. A centralized Salesforce application simplifies all of these in one place.

Key Requirements

- Gather requirements from customers, dealers and sales staff.
- Define system scope and user personas (Customer, Dealer, Admin).
- Design the data model with relationships between Vehicle, Dealer, Customer, Order, Test Drive and Service Request.
- Identify use cases: test drive booking, order management, service requests.
- Select tools: Salesforce Flows, Apex, reports and dashboards.

Project Scope & Objectives

- Build a scalable CRM solution that manages vehicles, dealers, customers, orders, test drives and service requests.
- Include data model design, automation (Flows, Apex, email alerts) and reporting dashboards.
- Provide role-based access for admins, dealers and customers.
- Improve customer experience with transparent order and service tracking.

Salesforce Features Planned

- **Custom objects:** Vehicle__c, Vehicle_Dealer__c, Vehicle_Customer__c, Vehicle_Order__c, Vehicle_Test_Drive__c, Vehicle_Service_Request__c
- **Automation:** record-triggered flows, screen flows, scheduled flows, approval processes
- **Apex:** classes and triggers for order/test drive logic; Batch Apex for periodic updates
- **UI:** WhatNext Vision Motors Lightning App, custom tabs, Lightning Record Pages
- **Email:** templates and alerts for booking, order and service confirmations
- **Security:** profiles, permission sets, role hierarchy, field-level security, sharing rules

Designing Data Model and Security Model

Object	Purpose and relationships
Vehicle__c	Stores core vehicle details such as name, Vehicle Model (picklist: Sedan, SUV, EV, etc.) and status. Linked to orders, test drives and service requests.
Vehicle_Dealer__c	Stores dealer name, location and contact details. Linked to vehicles, orders and test drives.
Vehicle_Customer__c	Stores customer name, contact and email. Linked to orders, test drives and service requests.
Vehicle_Order__c	Stores order details and status. Linked to Vehicle, Customer and Dealer.
Vehicle_Test_Drive__c	Stores test drive date, status and feedback. Linked to Vehicle, Customer and Dealer.
Vehicle_Service_Request__c	Stores service issue, status and resolution. Linked to Vehicle and Customer.

Security Model Design

- **Role hierarchy:** Administrator, Sales Manager, Dealer, Customer.
- **Profiles:** Admin - full access; Dealer - read/edit on vehicle, order and test drive objects; Customer - read-only access to own records; Support - access to service requests.

- **Permission sets:** grant extra privileges such as "Manage Orders".
- **Record-level security:** sharing rules on Vehicle_Order__c and Vehicle_Test_Drive__c for controlled dealer and manager access.
- **Field-level security:** sensitive customer contact fields restricted to Admin and Dealer profiles.
- *Note: object permissions are disabled by default for new custom objects, so enable them through permission sets or profiles.*

WhatNext Vision Motors

Field Creation, Automation, Creation

Vehicle Management & Test Drive Tracker (Salesforce)

Section	Covers
Part 1 - Field Creation	Fields and relationships on Vehicle, Vehicle Dealer and Vehicle Customer
Part 2 - Record Creation	Sample Dealer, Vehicle and Customer records entered in the app
Part 3 - Automation Creation	Auto Assign Dealer flow and Test Drive Reminder flow, errors and fixes
Part 4 - Testing & Summary	Checklist to verify everything works

Org: Salesforce Developer Edition | App: WhatNext Vision Motors | Date: 03 Oct 2026

1. Overview

This document explains, step by step, how the data fields, sample records and automation were built for the WhatNext Vision Motors Lightning App. The app stores vehicle, dealer and customer information in custom objects, and uses Flow Builder to automate dealer assignment and test drive reminders.

The screenshots taken during the build show three kinds of work:

- **Field creation** - fields such as Vehicle Model, Stock Quantity, Price, Dealer Code, Phone, Email and Preferred Vehicle Type.
- **Record creation** - entering sample records such as dealers Emm and PB, vehicle Yamaha and customer John.
- **Automation creation** - record-triggered flows named Auto Assign Dealer and a test drive reminder flow.

Figure 1. High-level architecture

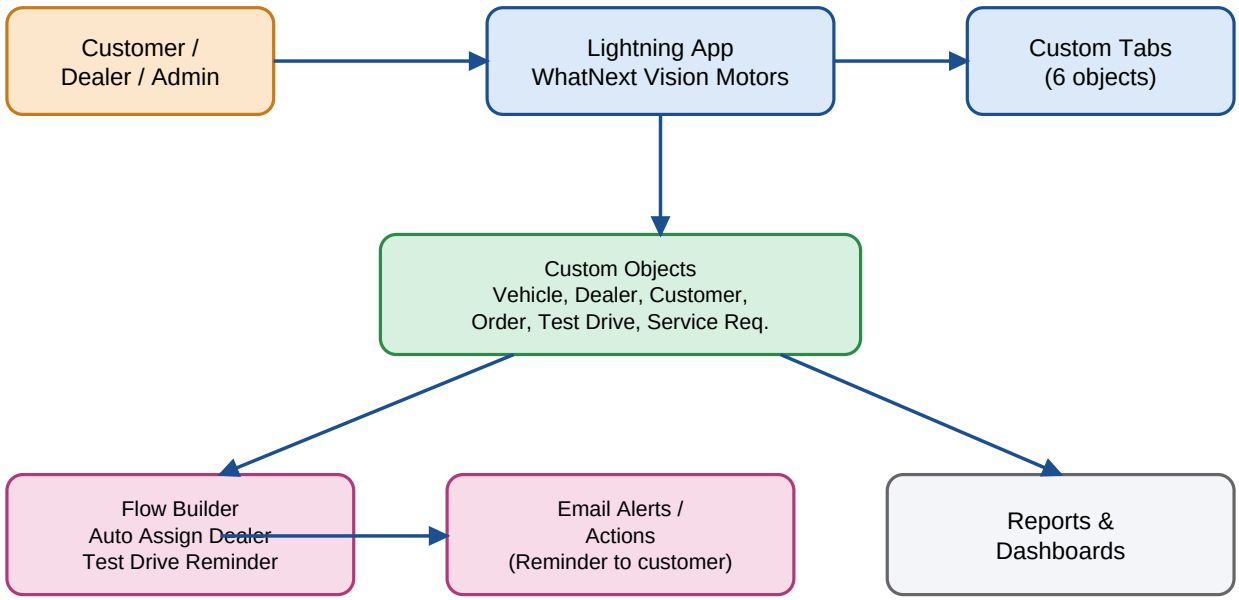
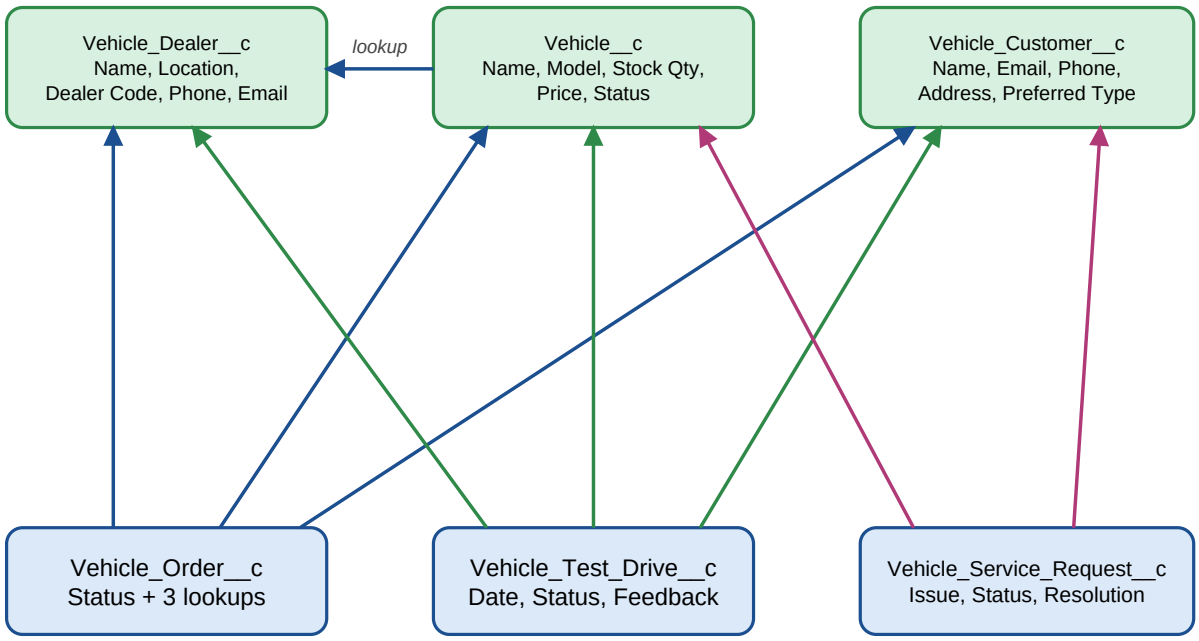


Figure 1. Users work in the Lightning App; tabs open the custom objects; flows automate actions on those objects.

2. Data Model

The solution uses six custom objects. Three parent objects (Vehicle Dealer, Vehicle, Vehicle Customer) hold master information. Three child objects (Order, Test Drive, Service Request) connect to those parents through lookup fields.

Order = blue lines | Test Drive = green | Service Request = pink



Each arrow is a lookup relationship from the child object to the parent object

Figure 2. Object relationship diagram

Object summary

Object (API name)	Role	Parent / lookup links
Vehicle_Dealer__c	Dealer master data	None (parent)
Vehicle__c	Vehicle stock and price	Vehicle Dealer (lookup)
Vehicle_Customer__c	Customer master data	None (parent)
Vehicle_Order__c	Customer vehicle orders	Vehicle, Customer, Dealer
Vehicle_Test_Drive__c	Test drive bookings	Vehicle, Customer, Dealer
Vehicle_Service_Request__c	After-sales service	Vehicle, Customer

PART 1 - FIELD CREATION

3. How to Create a Custom Field

Every field in this project is created the same way. Repeat the steps below for each field listed on the next pages.

1. Open **Setup** (gear icon) and go to **Object Manager**.
2. Select the object (for example **Vehicle**) and click **Fields & Relationships**.
3. Click **New**.
4. Choose the **Data Type** (Text, Number, Currency, Phone, Email, Picklist, Lookup, Auto Number) and click **Next**.
5. Enter the **Field Label**. The **Field Name** (API name) fills in automatically and ends with __c.
6. Set properties such as length, decimal places, required, or picklist values. Click **Next**.
7. Set **Field-Level Security** so the right profiles can see or edit the field. Click **Next**.
8. Add the field to the page layout and click **Save**.

Field data types used

Data type	Used for	Example value in this project
Text	Short names and addresses	Chennai, John
Number	Quantities	Stock Quantity = 150
Currency	Prices	$96,000
Phone / Email	Contact details	(963) 852-7410 / xyz@gmail.com
Picklist	Fixed choice lists	Vehicle Model = EV, Status = Available
Auto Number	System generated codes	DC-0001, DC-0002
Lookup	Links to another object	Vehicle Dealer on Vehicle

4. Vehicle (Vehicle__c) Fields

The Vehicle object stores each model the dealership sells. The record opened in the screenshot is a Yamaha EV.

Field label	Data type	Purpose	Sample value
Vehicle Name	Text (record name)	Name of the vehicle	Yamaha
Vehicle Model	Picklist	Vehicle category: Sedan, SUV, EV, etc.	EV
Stock Quantity	Number	Units available in stock	150
Price	Currency	Selling price	$96,000
Vehicle Dealer	Lookup (Vehicle Dealer)	Dealer who holds this vehicle	pg
Status	Picklist	Availability of the vehicle	Available

Steps for the picklist fields

1. On the Vehicle object, click **New** under Fields & Relationships and choose **Picklist**.
2. Label it **Vehicle Model** and enter one value per line: Sedan, SUV, EV (add more if needed).
3. Repeat for **Status** with values such as Available and Sold.
4. Create **Stock Quantity** as Number (no decimals) and **Price** as Currency.
5. Create **Vehicle Dealer** as a Lookup pointing to the Vehicle Dealer object.

Tip: create the Vehicle Dealer object and its fields first, so the lookup on Vehicle has a target object to choose.

5. Vehicle Dealer (Vehicle_Dealer__c) Fields

The Vehicle Dealer object holds each dealership. The two dealer records in the screenshots are Emm (Chennai) and PB (Kanchipuram).

Field label	Data type	Purpose	Sample values
Vehicle Dealer Name	Text (record name)	Dealer name	Emm, PB
Dealer Location	Text	City of the dealership	Chennai, Kanchipuram
Dealer Code	Auto Number (DC-{0000})	Unique code generated by Salesforce	DC-0001, DC-0002
Phone	Phone	Dealer contact number	(963) 852-7410, (789) 456-1230
Email	Email	Dealer contact email	xyz@gmail.com, abc@gmail.com

Why Dealer Code is an Auto Number

An Auto Number field gives every dealer a unique, ordered code without any manual typing. The first dealer saved received DC-0001 and the second received DC-0002, which confirms the numbering works. The display format used is `DC-{0000}` with a starting number of 1.

System fields shown on every record

- **Owner** - the user who owns the record (Bhavani s in the screenshots).
- **Created By** - user and time of creation (10/3/2026).
- **Last Modified By** - user and time of the last edit.

6. Vehicle Customer (Vehicle_Customer__c) Fields

The Vehicle Customer object holds buyer details. The customer record in the screenshots is John, who prefers a Sedan.

Field label	Data type	Purpose	Sample value
Vehicle Customer Name	Text (record name, required)	Customer name	John
Email	Email	Used for notifications and reminders	bhava1807@gmail.com
Phone	Phone	Contact number	9876543210 (shown as (987) 654-3210)
Address	Text	Customer city / address	Chennai
Preferred Vehicle Type	Picklist	Type of vehicle the customer wants	Sedan

Notes

- The **name** field is marked required, shown by the red asterisk on the New Vehicle Customer form.
- Salesforce formats a 10 digit Phone value automatically, so 9876543210 is displayed as (987) 654-3210.
- **Preferred Vehicle Type** uses the same list as Vehicle Model (Sedan, SUV, EV) so customers can be matched to vehicles.
- The **Email** field is important because the reminder flow sends its message to this address.

7. Relationship (Lookup) Fields

Lookup fields connect records. In the flow screenshot, the field picker shows relationship fields such as **Assigned Dealer**, **Vehicle** and **Vehicle Customer**. These come from lookups on the child objects.

On object	Lookup field	Points to
Vehicle	Vehicle Dealer	Vehicle Dealer
Vehicle Order	Vehicle	Vehicle
Vehicle Order	Vehicle Customer	Vehicle Customer
Vehicle Order	Assigned Dealer	Vehicle Dealer
Vehicle Test Drive	Vehicle / Vehicle Customer / Dealer	Vehicle, Vehicle Customer, Vehicle Dealer
Vehicle Service Request	Vehicle / Vehicle Customer	Vehicle, Vehicle Customer

Steps to create a Lookup field

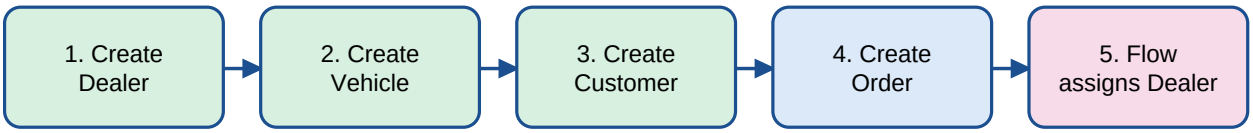
1. Object Manager > choose the child object (for example Vehicle Order) > Fields & Relationships > **New**.
2. Select data type **Lookup Relationship** and click Next.
3. Choose the related object (for example Vehicle Dealer) and click Next.
4. Enter the field label (for example **Assigned Dealer**) and click Next.
5. Set field-level security, add to the page layout and Save.

The **Assigned Dealer** field is the one the automation fills in. It is the key field that links an order to its dealer.

PART 2 - RECORD CREATION

8. Sample Records Created

After the fields were ready, sample records were entered through the app tabs. Click a tab, then **New**, fill in the form and **Save**. Parent records come first.



Parent records are created first so lookups always have something to point to

Figure 3. Order in which records are created

Records entered

Object	Record	Key values
Vehicle Dealer	Emm	Chennai, DC-0001, (963) 852-7410, xyz@gmail.com
Vehicle Dealer	PB	Kanchipuram, DC-0002, (789) 456-1230, abc@gmail.com
Vehicle	Yamaha	EV, Stock 150, $96,000, Dealer pg, Status Available
Vehicle Customer	John	bhava1807@gmail.com, 9876543210, Chennai, Sedan

All records were created by the user Bhavani s on 10/3/2026, who is also the record owner.

9. Creating a Customer Record - Walkthrough

The New Vehicle Customer window (Information section) is used as follows:

1. Open the **Vehicle Customer** tab. The list view says *Nothing to see here* when no records exist yet.
2. Click **New**. A window titled **New Vehicle Customer** opens.
3. Enter **vehicle customer Name** = John (required).
4. Enter **Email** = bhava1807@gmail.com.
5. Enter **Phone** = 9876543210.
6. Enter **Address** = Chennai.
7. Choose **Preferred Vehicle Type** = Sedan from the dropdown.
8. Click **Save**. Use **Save & New** to add another customer, or **Cancel** to close without saving.

The saved record opens on the **Details** tab and the **Related** tab lists connected records such as orders and test drives.

Dealer and Vehicle records

Dealer records (Emm and PB) and the Vehicle record (Yamaha) are created in the same way from the **Vehicle Dealers** and **vehicles** tabs. For the Yamaha record, the **Vehicle Dealer** lookup is filled by searching and selecting an existing dealer.

PART 3 - AUTOMATION CREATION

10. Automation Overview

Two record-triggered flows were built in Flow Builder (Setup > Flows > New Flow > Record-Triggered Flow). A record-triggered flow starts automatically when a record is created or updated.

Flow	Object	Purpose	Status in screenshots
Auto Assign Dealer	Vehicle Order	Finds the customer and nearest dealer and writes the dealer onto the order	Inactive (V1) at 02:48 PM, then Active at 03:11 PM
Test Drive Reminder	Vehicle Test Drive	Sends a reminder to the customer before the test drive	Built with a scheduled path

Flow elements used

Element	What it does
Start (Record-Triggered Flow)	Defines the object and when the flow runs
Run Immediately	Path that runs right after the record is saved
Scheduled path	Path that runs at a set time, for example before a test drive
Get Records	Looks up records (customer, dealer)
Update Records	Writes values onto a record (Assigned Dealer on the order)
Action	Calls an action such as sending an email reminder
End	Finishes the path

11. Flow 1 - Auto Assign Dealer

This flow assigns a dealer to a new vehicle order automatically, so staff do not have to pick one by hand.

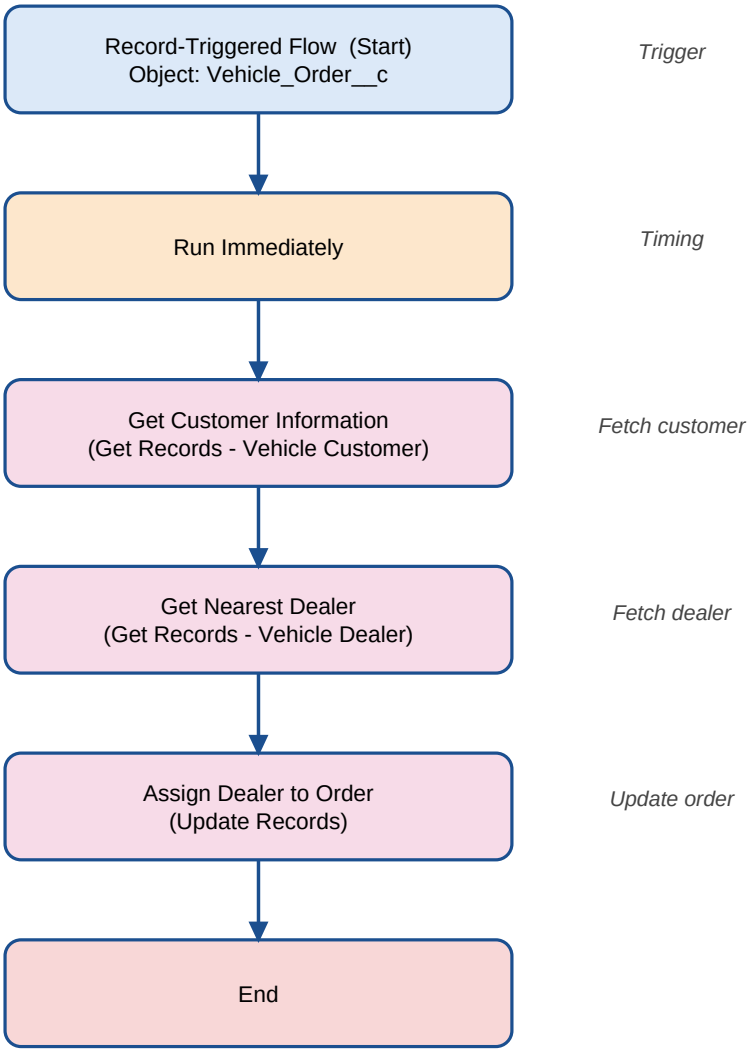


Figure 4. Auto Assign Dealer flow as seen in Flow Builder

12. Flow 1 - Element Configuration

Step	Element name	Type	Configuration
1	Start	Record-Triggered Flow	Runs when a Vehicle Order record is saved; path is Run Immediately
2	Get Customer Information	Get Records	Object: Vehicle Customer. Condition: Field **Record ID** Equals a value taken from the triggering record
3	Get Nearest Dealer	Get Records	Finds the dealer record that should serve the customer
4	Assign Dealer to Order	Update Records	Updates the order's **Assigned Dealer** field with the dealer found
5	End	End	Flow finishes

Configuring Get Customer Information

1. Click the **+** after Run Immediately and choose **Get Records**.
2. Label: **Get Customer Information**. The API name becomes `Get_Customer_Information`.
3. Under *Get Records of This Object*, set **Object** = Vehicle Customer.
4. Under *Filter Vehicle Customer Records*, keep **Condition Requirements** = All Conditions Are Met (AND).
5. Set **Field** = Record ID and **Operator** = Equals.
6. In the **Value** box, open the resource picker: All Resources > **Triggering Vehicle Order** record. The list shows relationship fields such as Assigned Dealer, Created By ID, Last Modified By ID, Owner ID, Vehicle and Vehicle Customer.
7. Choose the field that holds the customer (Vehicle Customer) so the flow finds the right customer record. The value box shows an expression starting with `{!$Record`.
8. Click **Done**, then **Save**.

13. Errors and Fixes

Two problems appeared while building and testing the flow.

Error 1 - Flow name already exists

When saving, the Errors and Warnings panel showed **Errors (1)** with an Automation Error: *A flow or process named Auto_Assign_Dealer-1 already exists.* Salesforce needs every flow to have a unique API name.

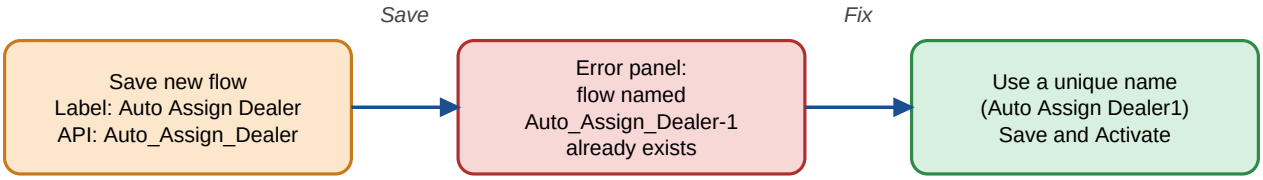


Figure 5. Duplicate name error and its fix

Fix

1. Open **Save As** or the flow properties and change the **Flow Label** and **Flow API Name** to something unique, here **Auto Assign Dealer1**.
2. Alternatively, delete or rename the older flow with the same name from Setup > Flows.
3. Save again. The panel then shows **You have 0 errors** and **Warnings (0)**.
4. Click **Activate**. The status badge changes from **Inactive** to **Active**.

Warning icon on Assign Dealer to Order

In the later screenshot a yellow warning triangle sits above **Assign Dealer to Order**. The filter value on Get Customer Information also reads like *Order > Last Modified Date*, which is a date field, while the filter field is Record ID (an ID). A Record ID should be compared with an ID such as the order's Vehicle Customer, not with a date. Change the Value to the customer lookup field and re-check the warning.

14. Flow 2 - Test Drive Reminder

This flow has two paths. Nothing extra happens immediately. The scheduled path waits and then reminds the customer before the test drive.

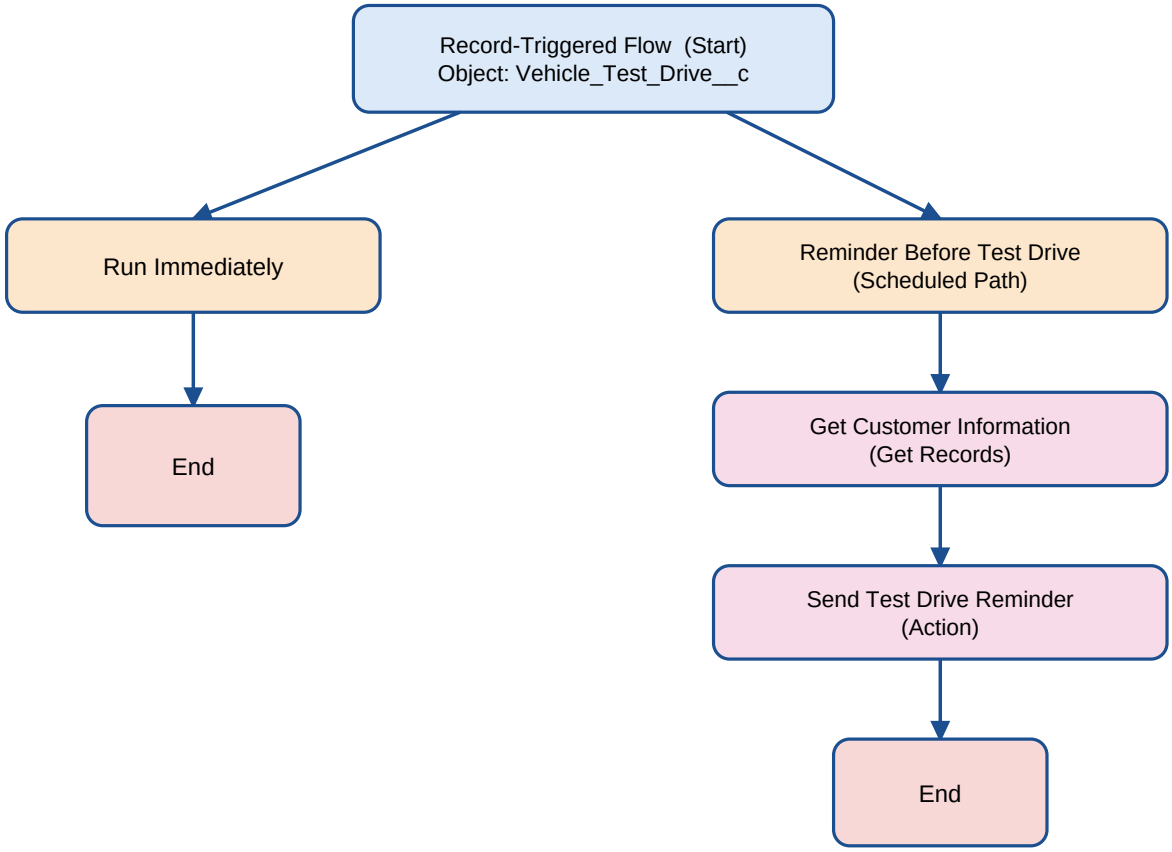


Figure 6. Test drive reminder flow with a scheduled path

Elements

Path	Element	Purpose
Run Immediately	End	No immediate action; flow ends
Reminder Before Test Drive	Get Customer Information	Finds the customer who booked the test drive
Reminder Before Test Drive	Send Test Drive Reminder	Action that sends the reminder using the customer's email
Reminder Before Test Drive	End	Flow finishes

15. Building the Scheduled Reminder

1. Setup > Flows > **New Flow** > **Record-Triggered Flow**.
2. Object: **Vehicle Test Drive**. Trigger: a record is created or updated.
3. Leave **Run Immediately** as is and click **Add Scheduled Paths**.
4. Name the new path **Reminder Before Test Drive** and set the time source to the test drive date with an offset (for example 1 day before).
5. On that path add **Get Records** named **Get Customer Information** (Object: Vehicle Customer).
6. Add an **Action** named **Send Test Drive Reminder** (Send Email or an email alert using a template).
7. Add **End** on both paths, then **Save** and **Activate**.

Email reminder - suggested template

Item	Content
Subject	Reminder: your test drive is coming up
Body	Hello {customer name}, this is a reminder about your test drive with WhatNext Vision Motors. Please reach us if you need to reschedule. Thank you.
To	Customer Email field on the Vehicle Customer record

PART 4 - TESTING & SUMMARY

16. Testing Checklist

No.	What to test	Expected result
1	Create dealers Emm and PB	Dealer Codes DC-0001 and DC-0002 are generated
2	Create vehicle Yamaha with a Vehicle Dealer	Record saves with Model EV, Stock 150, Price $96,000
3	Create customer John	Phone is formatted as (987) 654-3210
4	Save a flow with a duplicate name	Error appears; unique name removes it
5	Activate Auto Assign Dealer1	Badge shows Active, 0 errors, 0 warnings
6	Create a Vehicle Order	Assigned Dealer fills in automatically
7	Create a Test Drive and wait for the schedule	Customer receives the reminder email

Debugging tips

- Use the **Debug** button in Flow Builder to run the flow on a sample record and see each element's result.
- Open **Setup > Flows** and check that the correct version is Active.
- If a Get Records element returns nothing, check the filter condition and that the lookup field on the triggering record is filled.
- Confirm the user's profile has permission on all custom objects and fields, otherwise tabs and fields stay hidden.

17. Summary

What was completed

- Created fields on the Vehicle, Vehicle Dealer and Vehicle Customer objects, including picklists, a currency field, a number field, an auto number and lookups.
- Entered sample records: dealers Emm and PB, vehicle Yamaha, and customer John.
- Built the **Auto Assign Dealer** flow with Get Records and Update Records elements, resolved the duplicate name error and activated it.
- Built a **Test Drive Reminder** flow with a scheduled path that sends a reminder to the customer.

Key learnings

- Create parent objects and records before child records so lookups work.
- Flow names and API names must be unique.
- A Record ID filter must be compared with an ID value, not a date.
- Always check the Errors and Warnings panel before activating a flow.

Next steps

- Add validation rules to keep Stock Quantity and Price valid.
- Create reports and a dashboard on orders, test drives and stock by dealer.
- Add permission sets so Dealer and Customer profiles see only the data they need.

WhatNext Vision Motors

Apex and Trigger Batch Job Coding

Apex Trigger, Handler Class, Batch Apex and Scheduler (Salesforce)

Part	Covers
Part 1 - Overview	What the code does and how the pieces connect
Part 2 - Apex Trigger	VehicleOrderTrigger and VehicleOrderTriggerHandler
Part 3 - Batch Job	VehicleOrderBatch and VehicleOrderBatchScheduler
Part 4 - Testing	Steps, results seen in the org, notes and summary

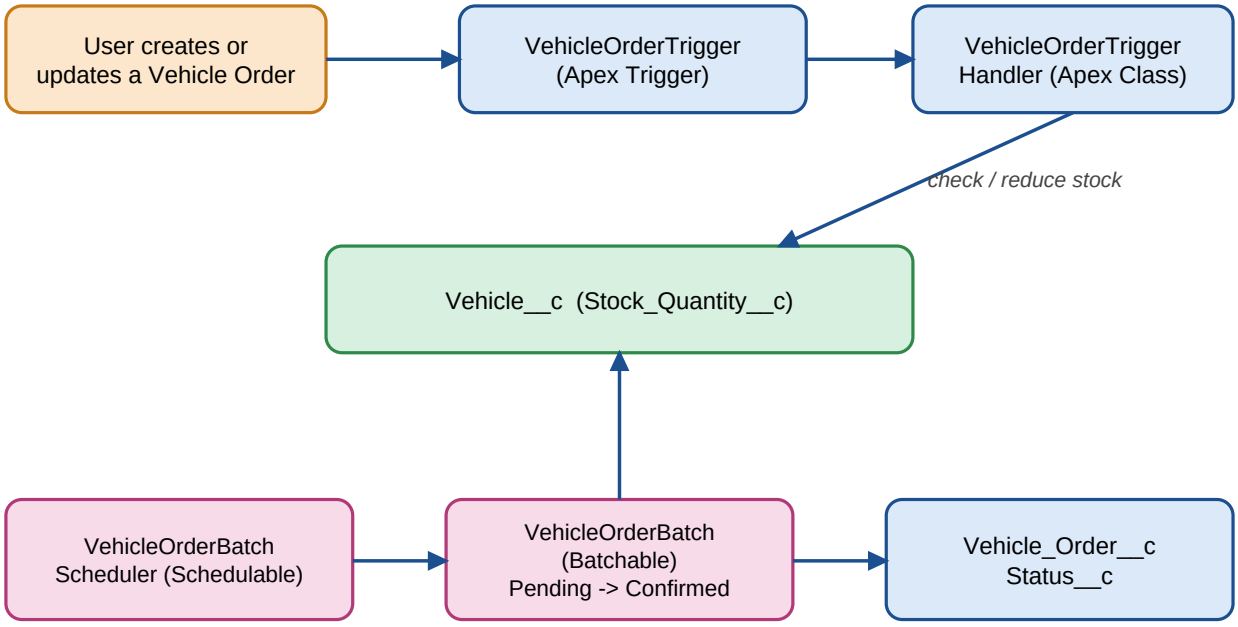
Org: Salesforce Developer Edition | Tool: Developer Console | API Version 67 | Date: 03 Oct 2026

1. Overview

The Vehicle Order process needs two rules that Flows alone handle poorly: stop an order when a vehicle is out of stock, and reduce stock when an order is confirmed. These are written in Apex. A second piece, a batch job, runs on a schedule and confirms every **Pending** order that still has stock.

The code has four parts, all created in the Developer Console:

- **VehicleOrderTrigger** - an Apex trigger on Vehicle_Order__c.
- **VehicleOrderTriggerHandler** - an Apex class holding the logic, so the trigger stays small.
- **VehicleOrderBatch** - Batch Apex that confirms Pending orders.
- **VehicleOrderBatchScheduler** - a Schedulable class that starts the batch at set times.



Top: real-time logic (trigger) | Bottom: scheduled logic (batch job)

Figure 1. How the trigger, batch job and objects connect

2. Components and Fields Used

Component	Type	File in Developer Console	Purpose
VehicleOrderTrigger	Apex Trigger	VehicleOrderTrigger.apxt	Fires on insert and update of an order
VehicleOrderTriggerHandler	Apex Class	VehicleOrderTriggerHandler.apxc	Stock check and stock update logic
VehicleOrderBatch	Batch Apex	VehicleOrderBatch.apxc	Confirms Pending orders in chunks
VehicleOrderBatchScheduler	Schedulable	VehicleOrderBatchScheduler.apxc	Starts the batch (batch size 50)

Fields the code reads or changes

Object	Field (API name)	Used for
Vehicle_Order__c	Vehicle__c	Lookup to the ordered vehicle
Vehicle_Order__c	Status__c	Pending or Confirmed; the batch and trigger check this
Vehicle__c	Stock_Quantity__c	Units in stock; checked and reduced by the code

Before you start

- Create the Vehicle and Vehicle Order objects and fields first (see the earlier field creation guide).
- Make sure **Status__c** has the values **Pending** and **Confirmed**.
- Make sure **Stock_Quantity__c** is a Number field on Vehicle__c.
- Apex classes and triggers can be written in a Developer Edition org without extra setup.

3. Creating Apex in the Developer Console

All four files were created in the Developer Console (opened from the gear icon > Developer Console). The screenshots show the Console with API Version 67 and Code Coverage set to None.

Create an Apex class

1. In Developer Console choose **File > New > Apex Class**.
2. Enter the class name, for example **VehicleOrderTriggerHandler**, and click OK.
3. Paste the code and press **Ctrl + S** to save. Errors, if any, appear in the Problems tab.

Create an Apex trigger

1. Choose **File > New > Apex Trigger**.
2. In the *New Apex Trigger* window enter **Name** = VehicleOrderTrigger.
3. Choose **sObject** = Vehicle_Order__c from the dropdown.
4. Click **Submit**, replace the template with the trigger code and save.

Item in screenshot	Value
Open editor tabs	VehicleOrderTriggerHandler.apxc, VehicleOrderTrigger.apxt, VehicleOrderBatch.apxc, VehicleOrderBatchScheduler.apxc
New Apex Trigger dialog	Name: VehicleOrderTrigger, sObject: Vehicle_Order__c
API Version	67
Bottom panel tabs	Logs, Tests, Checkpoints, Query Editor, View State, Progress, Problems

PART 2 - APEX TRIGGER

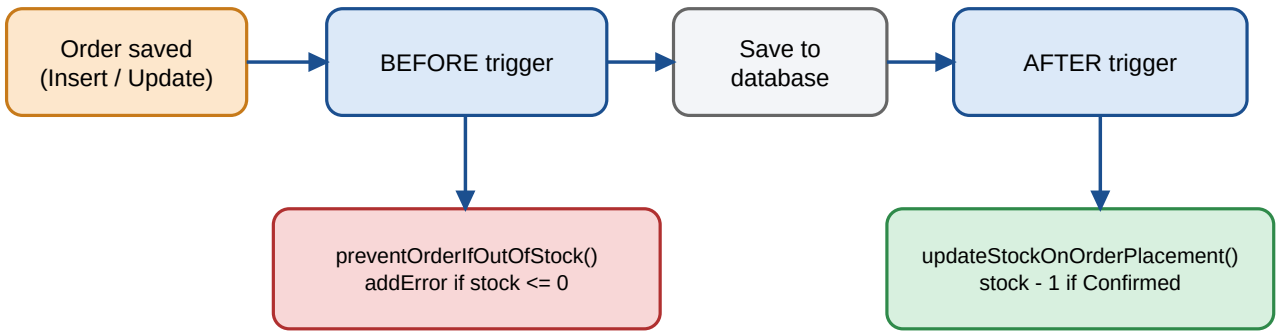
4. VehicleOrderTrigger

The trigger listens for four events on Vehicle_Order__c and passes everything to the handler class. Keeping the trigger thin makes the logic easier to read, reuse and test.

```
trigger VehicleOrderTrigger on Vehicle_Order__c
        (before insert, before update, after insert, after update) {

    VehicleOrderTriggerHandler.handleTrigger(
        Trigger.new, Trigger.oldMap,
        Trigger.isBefore, Trigger.isAfter,
        Trigger.isInsert, Trigger.isUpdate
    );
}
```

Figure 2 shows when each part runs.



before = validate the data | after = update other records

Figure 2. Trigger events and the handler methods they call

What the context variables mean

Variable	Meaning
Trigger.new	List of new or changed order records
Trigger.oldMap	Map of the old values (empty on insert)
Trigger.isBefore / isAfter	Whether the trigger runs before or after the record is saved
Trigger.isInsert / isUpdate	Which operation fired the trigger

5. VehicleOrderTriggerHandler - handleTrigger

The handler's public method decides which private method to run.

```
public class VehicleOrderTriggerHandler {

    public static void handleTrigger(
            List<Vehicle_Order__c> newOrders,
            Map<Id, Vehicle_Order__c> oldOrders,
            Boolean isBefore, Boolean isAfter,
            Boolean isInsert, Boolean isUpdate) {

        if (isBefore && (isInsert || isUpdate)) {
            preventOrderIfOutOfStock(newOrders);
        }

        if (isAfter && (isInsert || isUpdate)) {
            updateStockOnOrderPlacement(newOrders);
        }
    }
    // ... private methods continue on the next pages
}
```

Condition	Method called	When
isBefore AND (insert OR update)	preventOrderIfOutOfStock	Before the order is saved; can block it
isAfter AND (insert OR update)	updateStockOnOrderPlacement	After the order is saved; updates the vehicle

Why before and after?

- **Before** triggers are used for validation. `addError()` in a before trigger stops the save.
- **After** triggers are used to change other records, here the Vehicle. The order already has an Id and a saved state at that point.

Note: the handler receives `oldOrders` but the current methods do not use it yet. See the notes page for how it can help.

6. preventOrderIfOutOfStock

This method blocks an order when its vehicle has no stock left.

```
// Prevent placing an order if stock is zero
private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
    Set<Id> vehicleIds = new Set<Id>();
    for (Vehicle_Order__c order : orders) {
        if (order.Vehicle__c != null) {
            vehicleIds.add(order.Vehicle__c);
        }
    }

    if (!vehicleIds.isEmpty()) {
        Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>(
            [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]
        );

        for (Vehicle_Order__c order : orders) {
            Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
            if (vehicle != null && vehicle.Stock_Quantity__c <= 0) {
                order.addError('This vehicle is out of stock. Order cannot be placed.');
            }
        }
    }
}
```

Logic flow - preventOrderIfOutOfStock

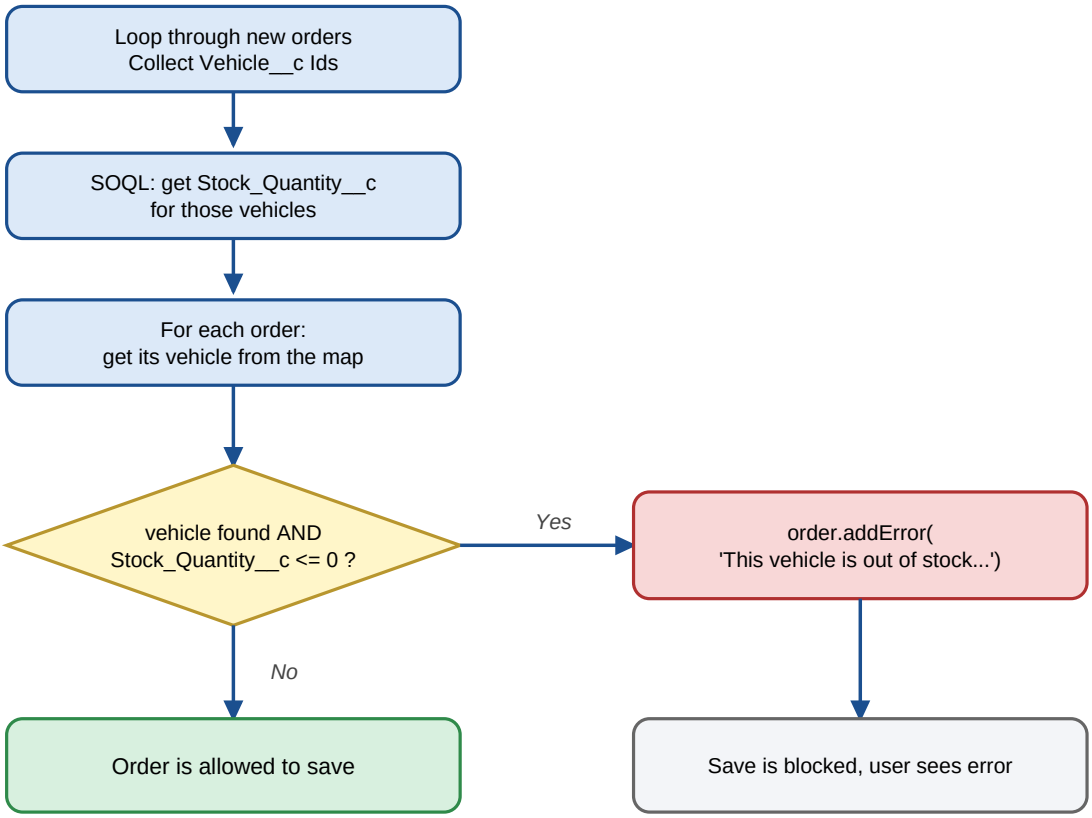


Figure 3. Out of stock check

How it works

1. Collect the Vehicle__c Ids from all orders in the transaction.
2. Run **one** SOQL query to get stock for all those vehicles (bulk safe, no query inside a loop).
3. For each order, look up its vehicle in the map.
4. If stock is 0 or less, call `order.addError(...)`. The user sees the message and the order is not saved.

7. updateStockOnOrderPlacement

This method reduces the vehicle's stock by one when an order has the status **Confirmed**.

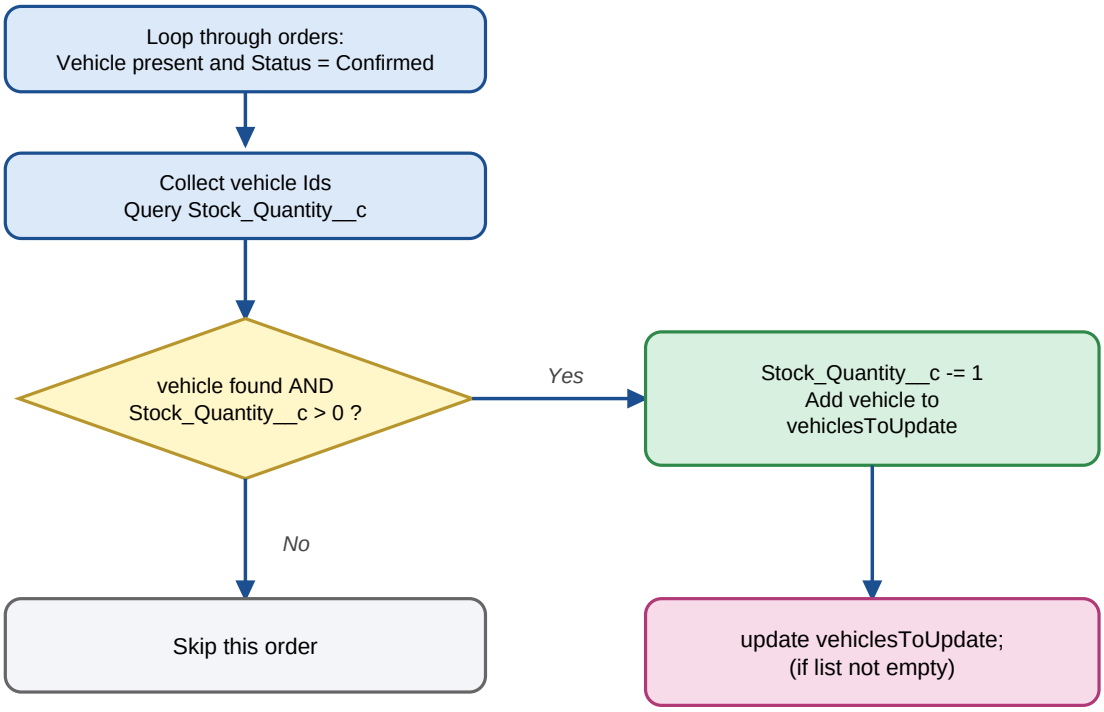
```
// Decrease stock when an order is confirmed
private static void updateStockOnOrderPlacement(List<Vehicle_Order__c> orders) {
    Set<Id> vehicleIds = new Set<Id>();
    for (Vehicle_Order__c order : orders) {
        if (order.Vehicle__c != null && order.Status__c == 'Confirmed') {
            vehicleIds.add(order.Vehicle__c);
        }
    }

    if (!vehicleIds.isEmpty()) {
        Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>(
            [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]
        );

        List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
        for (Vehicle_Order__c order : orders) {
            Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
            if (vehicle != null && vehicle.Stock_Quantity__c > 0) {
                vehicle.Stock_Quantity__c -= 1;
                vehiclesToUpdate.add(vehicle);
            }
        }

        if (!vehiclesToUpdate.isEmpty()) {
            update vehiclesToUpdate;
        }
    }
}
```

Logic flow - updateStockOnOrderPlacement



Result: one unit is removed from stock for each confirmed order

Figure 4. Stock reduction after a confirmed order

Key points

- Only vehicles from orders with `Status__c == 'Confirmed'` are queried.
- A single DML `update` is used for the whole list, which respects Salesforce governor limits.
- The stock is only reduced when it is greater than zero, so it cannot go negative.

PART 3 - BATCH JOB

8. VehicleOrderBatch

Batch Apex processes large numbers of records in small chunks. This batch finds all **Pending** orders and confirms the ones whose vehicle still has stock. The class implements `Database.Batchable<sObject>` and has three methods: **start**, **execute** and **finish**.

Part 1 - start() and the beginning of execute()

```
global class VehicleOrderBatch implements Database.Batchable<sObject> {

    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator([
            SELECT Id, Status__c, Vehicle__c
            FROM Vehicle_Order__c
            WHERE Status__c = 'Pending'
        ]);
    }

    global void execute(Database.BatchableContext bc, List<Vehicle_Order__c> orderList) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orderList) {
            if (order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
        }

        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>(
                [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]
            );
            // ... continues on the next page
```

Part 2 - rest of execute() and finish()

```
            List<Vehicle_Order__c> ordersToUpdate = new List<Vehicle_Order__c>();
            List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();

            for (Vehicle_Order__c order : orderList) {
                Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                if (vehicle != null && vehicle.Stock_Quantity__c > 0) {
                    order.Status__c = 'Confirmed';
                    vehicle.Stock_Quantity__c -= 1;
                    ordersToUpdate.add(order);
                    vehiclesToUpdate.add(vehicle);
                }
            }

            if (!ordersToUpdate.isEmpty()) update ordersToUpdate;
            if (!vehiclesToUpdate.isEmpty()) update vehiclesToUpdate;
        }
    }

    global void finish(Database.BatchableContext bc) {
        System.debug('Vehicle order batch job completed.');
    }
}
```

Batch methods explained

Method	Runs	What it does here
start()	Once at the beginning	Returns a QueryLocator of all Pending orders
execute()	Once per chunk of records	Confirms orders and reduces stock where available
finish()	Once at the end	Writes a debug message: Vehicle order batch job completed.

Batch job flow

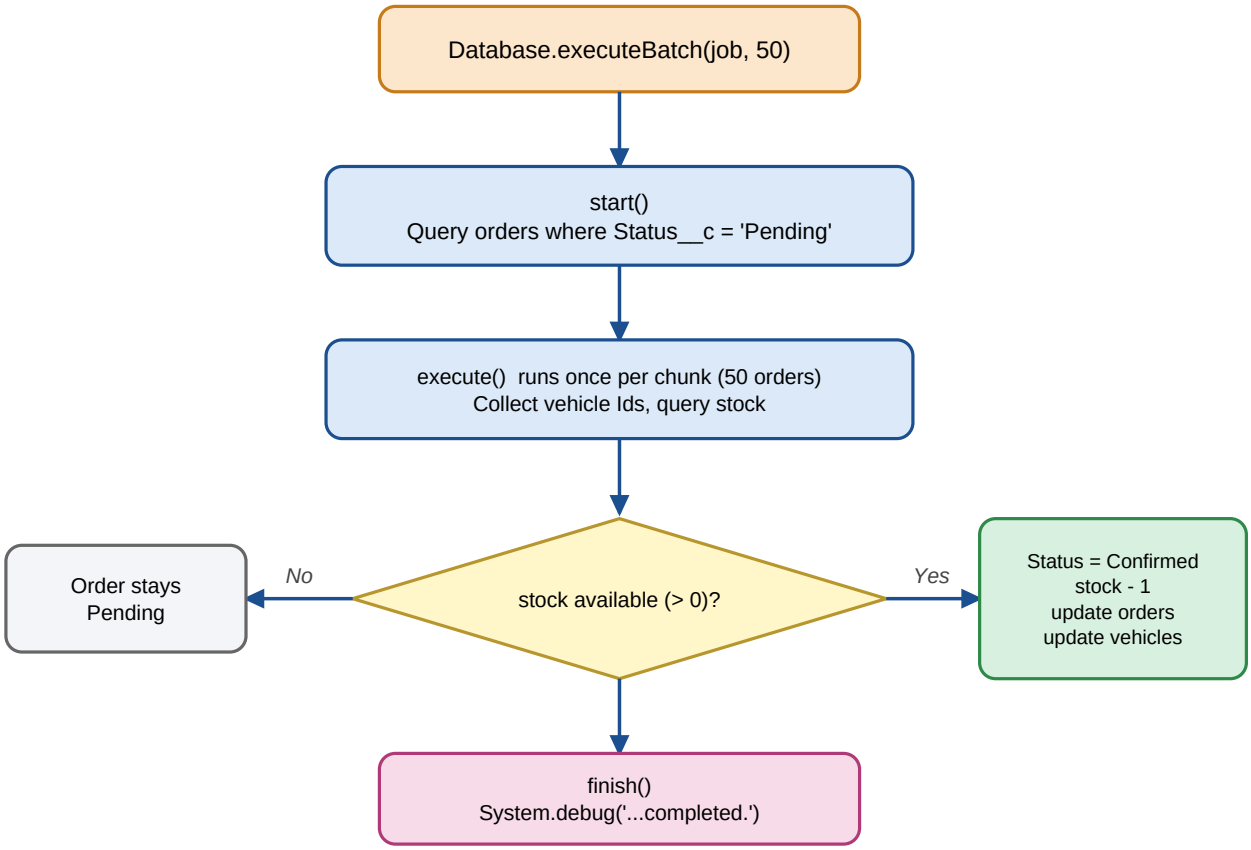


Figure 5. Execution flow of VehicleOrderBatch

Each chunk is its own transaction with fresh governor limits, which is why batch Apex can handle very large data volumes.

9. VehicleOrderBatchScheduler

The scheduler class starts the batch automatically. It implements `Schedulable`, and its `execute` method launches the batch with a batch size of 50.

```
global class VehicleOrderBatchScheduler implements Schedulable {
    global void execute(SchedulableContext sc) {
        VehicleOrderBatch batchJob = new VehicleOrderBatch();
        Database.executeBatch(batchJob, 50); // 50 = batch size
    }
}
```

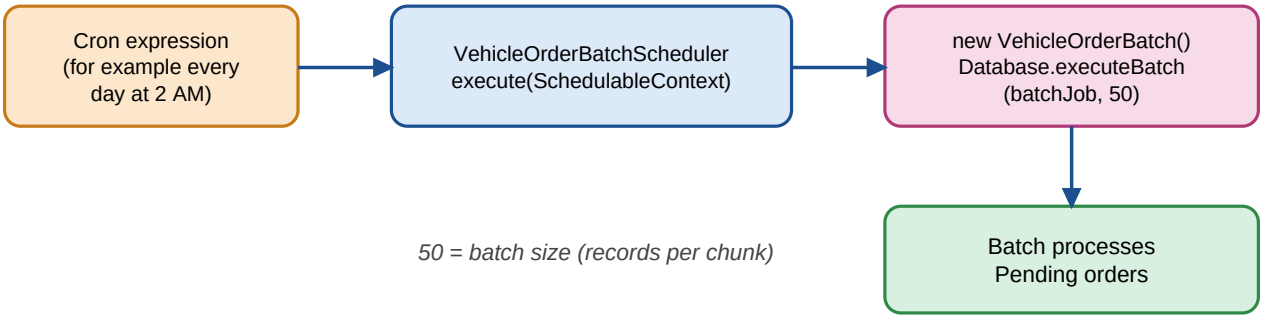


Figure 6. From schedule to batch run

Run or schedule the job

Open **Debug > Open Execute Anonymous Window** and run one of these:

```
// Run the batch once, right now (Developer Console > Debug > Open Execute Anonymous Window)
Database.executeBatch(new VehicleOrderBatch(), 50);

// Schedule it every day at 2:00 AM
String cron = '0 0 2 * * ?';
System.schedule('Vehicle Order Batch Daily', cron, new VehicleOrderBatchScheduler());
```

You can also schedule it from **Setup > Apex Classes > Schedule Apex**, and watch progress in **Setup > Apex Jobs** or **Scheduled Jobs**.

PART 4 - TESTING

10. How to Test

Test the trigger first, then the batch job.

No.	Test	Expected result
1	Create an order for a vehicle with Stock_Quantity__c = 0	Error: This vehicle is out of stock. Order cannot be placed.
2	Create an order with Status = Confirmed for a vehicle with stock	Order saves; vehicle stock goes down by 1
3	Create an order with Status = Pending	Order saves; stock unchanged
4	Run the batch (Execute Anonymous)	Pending orders become Confirmed; stock reduces
5	Open Setup > Apex Jobs	Job shows Completed
6	Open Developer Console > Logs	Entries show Success; debug text appears in the log

Result seen in the org

The Vehicle record **Yamaha** (Model EV, Price $96,000, Dealer PB, Status Available) had a stock of **150** in the earlier screenshot. After an order was confirmed, the record shows **149**, so the stock reduction worked.

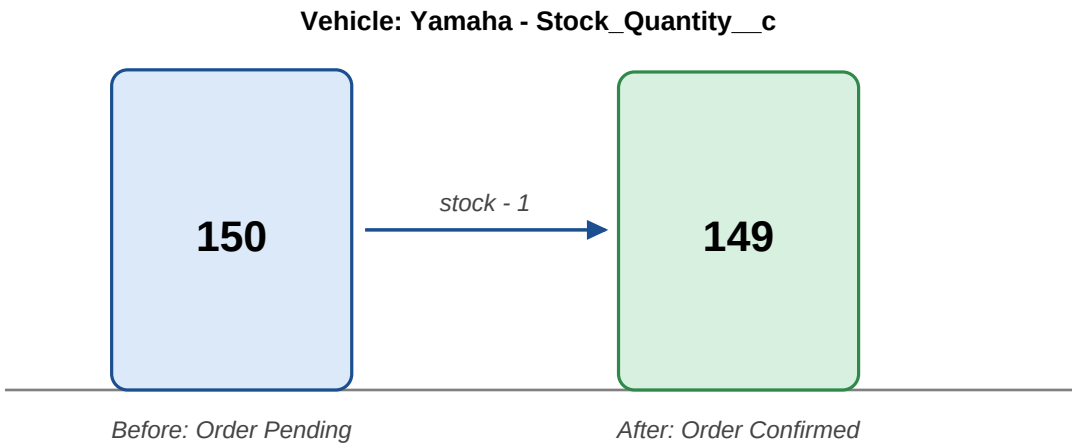


Figure 7. Stock before and after one confirmed order

11. Logs and Debugging

The Logs tab in Developer Console records each operation. The screenshot of the scheduler class shows two log rows:

User	Application	Operation	Time	Status	Size
Bhavani s	Browser	/aura	03/10/2026, 17:54:45	Success	17.58 KB
Bhavani s	Browser	/aura	03/10/2026, 17:51:29	Success	426 bytes

Debugging tips

- Double click a log to open it, then tick **Debug Only** to see only `System.debug` lines.
- Use the **Problems** tab to find compile errors such as a wrong field API name.
- Use **Query Editor** to check stock: `SELECT Name, Stock_Quantity__c FROM Vehicle__c`.
- Use **Setup > Apex Jobs** to see how many batches ran and whether any failed.
- Before moving to production, write test classes; Salesforce requires at least 75% code coverage. The Console currently shows Code Coverage: None.

12. Notes and Possible Improvements

The code works for the basic case shown in the screenshots. These points are worth knowing as the project grows.

Point	Why it matters	Suggestion
Trigger runs on every update	Any later edit of an already Confirmed order runs the stock reduction again	Compare with oldOrders and reduce stock only when Status changes to Confirmed
Two orders, same vehicle, one save	The same Vehicle record is added to the update list twice, which can raise a Duplicate id in list error	Count orders per vehicle in a Map, then update each vehicle once
Blank stock value	Comparing a null Stock_Quantity__c with 0 can cause an error	Treat null as 0, or make the field required
Batch also fires the trigger	The batch updates orders to Confirmed, which fires the trigger that also reduces stock, so stock is handled in two places	Keep the stock logic in one place only

Suggested version of the stock update method (not part of the screenshots)

```
private static void updateStockOnOrderPlacement(
        List<Vehicle_Order__c> orders, Map<Id, Vehicle_Order__c> oldOrders) {

    Map<Id, Integer> countByVehicle = new Map<Id, Integer>();
    for (Vehicle_Order__c o : orders) {
        Boolean wasConfirmed = oldOrders != null && oldOrders.containsKey(o.Id)
            && oldOrders.get(o.Id).Status__c == 'Confirmed';
        if (o.Vehicle__c != null && o.Status__c == 'Confirmed' && !wasConfirmed) {
            Integer c = countByVehicle.containsKey(o.Vehicle__c)
                ? countByVehicle.get(o.Vehicle__c) : 0;
            countByVehicle.put(o.Vehicle__c, c + 1);
        }
    }
    if (countByVehicle.isEmpty()) return;

    List<Vehicle__c> toUpdate = [SELECT Id, Stock_Quantity__c FROM Vehicle__c
                                 WHERE Id IN :countByVehicle.keySet()];
    for (Vehicle__c v : toUpdate) {
        Decimal stock = (v.Stock_Quantity__c == null) ? 0 : v.Stock_Quantity__c;
        v.Stock_Quantity__c = Math.max(0, stock - countByVehicle.get(v.Id));
    }
    update toUpdate;
}
```

If you use this version, pass `oldOrders` from handleTrigger when you call the method.

13. Summary

What was built

- **VehicleOrderTrigger** on Vehicle_Order__c for before insert, before update, after insert and after update.
- **VehicleOrderTriggerHandler** that blocks out of stock orders and reduces stock for confirmed orders.
- **VehicleOrderBatch** that confirms Pending orders in chunks and reduces stock.
- **VehicleOrderBatchScheduler** that starts the batch with a batch size of 50.

Key learnings

- Use before triggers for validation and after triggers for updating other records.
- Keep trigger logic in a handler class and avoid SOQL or DML inside loops.
- Batch Apex has three methods: start, execute and finish.
- A Schedulable class lets the batch job run automatically on a cron schedule.
- Test with Developer Console logs and Setup > Apex Jobs.

Next steps

- Write Apex test classes to reach 75% or more coverage.
- Apply the improvement notes so stock is reduced exactly once per confirmed order.
- Add a dashboard of orders by status and stock by vehicle.